AI ASSISTED CODING
ASSIGNEMNT -9.5

B-32
A. Niteesh Kumar
2303A52341

Problem 1: String Utilities Function
Consider the following Python function:
def reverse_string(text):
return text[::-1]
Task:
1. Write documentation in:
o (a) Docstring
o (b) Inline comments
o (c) Google-style documentation
2. Compare the three documentation styles.
3. Recommend the most suitable style for a utility-based string library.

CODE:

```
def reverse_string(s:str)-> str:
    """
    this function revreses the given string and returns it
    Args:
    s:the string to be converted
    returns: the reversed string
    """
    return s[::-1] # this is a slicing technique to reverse the string
```

Aspect	Docstring	Inline Comments	Google-Style Doc
Readability	High	Medium	Very High
Explains logic	Low	High	Medium
Explains parameters	Low	No	High
Tool support (IDE/docs)	Yes	No	Yes
Suitable for libraries	Medium	Low	High

Recommendation: Use Google-style docstrings + minimal inline comments only when logic is non-obvious.

Problem 2: Password Strength Checker
Consider the function:
def check_strength(password):
return len(password) >= 8
Task:
1. Document the function using docstring, inline comments, and Google style.
2. Compare documentation styles for security-related code.
3. Recommend the most appropriate style.

CODE:

```
def check_password_strength(password: str) -> str:
    """Check the strength of a given password.
    Args:
```

```
    password (str): The password to check.
       Returns:
       str: A message indicating the strength of the password.
       """
    if len(password) < 6:
        return "Weak password: Password should be at least 6 characters long."
    elif len(password) < 12:
        return "Moderate password: Consider adding more characters for better security."
    else:
        return "Strong password: Your password is strong."
```

Aspect	Docstring	Inline Comments	Google-Style Doc
Clarity	Medium	High (logic only)	Very High
Explains security rules	Low	High	High
Describes inputs/outputs	Low	No	Yes
Maintainability	Medium	Low	High
Suitable for security code	Medium	Medium	Best

Recommendation: Google-Style Documentation

Problem 3: Math Utilities Module
Task:
1. Create a module math_utils.py with functions:
o square(n)
o cube(n)
o factorial(n)
2. Generate docstrings automatically using AI tools.
3. Export documentation as an HTML file.
CODE:

```
def square(n: int) -> int:
    """Calculate the square of a number.
    Args:n (int): The number to square.
    Returns:
    int: The square of the number."""
    return n * n # This multiplies the number by itself to get the square.
def cube(n: int) -> int:
    """Calculate the cube of a number.
    Args:
    n (int): The number to cube.
    Returns:
    int: The cube of the number.
    """
    return n * n * n # This multiplies the number by itself three times to get the cube.

def factorial(n: int) -> int:
    """Calculate the factorial of a number.
    Args:
    n (int): The number to calculate the factorial of.
    Returns:
    int: The factorial of the number."""
    if n == 0:
        return 1 # Base case: factorial of 0 is 1
    else:
        return n * factorial(n - 1) # Recursive case: n! = n * (n - 1)!
print(square.__doc__)
print(cube.__doc__)
print(factorial.__doc__)
```

Output:

index
math_utils c:\users\ankam\onedrive\documents\python\math_utils.py

Functions

cube(n: int) -> int
Calculate the cube of a number.
Args:
n (int): The number to cube.
Returns:
int: The cube of the number.

factorial(n: int) -> int
Calculate the factorial of a number.
Args:
n (int): The number to calculate the factorial of.
Returns:
int: The factorial of the number.

square(n: int) -> int
Calculate the square of a number.
Args:n (int): The number to square.
Returns:
int: The square of the number.

Problem 4: Attendance Management Module
Task:
1. Create a module attendance.py with functions:
o mark_present(student)
o mark_absent(student)
o get_attendance(student)
2. Add proper docstrings.
3. Generate and view documentation in terminal and browse
CODE:

```
def mark_present(student: str, attendance: dict) -> None:
    """Marks a student as present in the attendance dictionary.

    Args:
        student: The name of the student to mark as present.
        attendance: A dictionary where keys are student names and values are their attendance status.
    """
    attendance[student] = 'Present'
def mark_absent(student: str, attendance: dict) -> None:
    """Marks a student as absent in the attendance dictionary.
    Args:
        student: The name of the student to mark as absent.
        attendance: A dictionary where keys are student names and values are their attendance status.
    """
    attendance[student] = 'Absent'
def get_attendance(student: str, attendance: dict) -> str:
    """Retrieves the attendance status of a student from the attendance dictionary.
    Args:
        student: The name of the student whose attendance status is to be retrieved.
        attendance: A dictionary where keys are student names and values are their attendance status.
    Returns:
        The attendance status of the student, or 'Unknown' if the student is not in the
        attendance dictionary.
    """
    return attendance.get(student, 'Unknown')
```

OUTPUT:

```
Help on module Documentation:

NAME
    Documentation

FUNCTIONS
    get_attendance(student: str, attendance: dict) -> str
        Retrieves the attendance status of a student from the attendance dictionary.

        Args:
            student: The name of the student whose attendance status is to be retrieved.
            attendance: A dictionary where keys are student names and values are their attendance status.
        Returns:
            The attendance status of the student, or 'Unknown' if the student is not in the
            attendance dictionary.

    mark_absent(student: str, attendance: dict) -> None
        Marks a student as absent in the attendance dictionary.

        Args:
            student: The name of the student to mark as absent.
            attendance: A dictionary where keys are student names and values are their attendance status.

    mark_present(student: str, attendance: dict) -> None
        Marks a student as present in the attendance dictionary.

        Args:
            student: The name of the student to mark as present.
            attendance: A dictionary where keys are student names and values are their attendance status.

FILE
    c:\users\ankam\onedrive\documents\python\documentation.py
```

Python 3.13.5 [tags/v3.13.5:6cb20a2, MSC v.1943 64 bit (AMD64)]
Windows-11

Documentation

Functions

get_attendance(student: str, attendance: dict) -> str

```
Retrieves the attendance status of a student from the attendance dictionary.

Args:
    student: The name of the student whose attendance status is to be retrieved.
    attendance: A dictionary where keys are student names and values are their attendance status.
Returns:
    The attendance status of the student, or 'Unknown' if the student is not in the
    attendance dictionary.
```

mark_absent(student: str, attendance: dict) -> None

```
Marks a student as absent in the attendance dictionary.

Args:
    student: The name of the student to mark as absent.
    attendance: A dictionary where keys are student names and values are their attendance status.
```

mark_present(student: str, attendance: dict) -> None

```
Marks a student as present in the attendance dictionary.

Args:
    student: The name of the student to mark as present.
    attendance: A dictionary where keys are student names and values are their attendance status.
```

Problem 5: File Handling Function
Consider the function:
def read_file(filename):
with open(filename, 'r') as f:
return f.read()
Task:
1. Write documentation using all three formats.
2. Identify which style best explains exception handling.
3. Justify your recommendation.

CODE:

Recommended Style: Google-Style Documentation

Justification:

File handling is error-prone (missing files, permission issues) Google-style documentation:

Clearly explains exceptions

Improves code reliability and usability

Helps developers handle errors correctly

```
def read_file(file_path: str) -> str:
     """Reads the content of a file and returns it as a string. param file_path:
     The path to the file to be read.
     return: The content of the file as a string.
     Exceptions: Raises FileNotFoundError if the file does not exist. """
     try:
          with open(file_path, 'r') as file: content
               = file.read()
          return content
     except FileNotFoundError:
          raise FileNotFoundError(f"The file at {file_path} was not found.")
read_file('example.txt')
```

Is widely used in production and open-source projects